
Software Testing Lab: Mastering Jest (Beginner to Advanced)

We are diving into software testing using **Jest**, a powerful and widely-used JavaScript testing framework. Testing ensures our code behaves as expected, prevents regressions, and serves as a form of living documentation.

This lab is split into two levels. You will build a simple utility tool, and add more complex integrations.

Environment Setup & Installation

Before we write any code, we need to initialize our Node.js environment and install Jest. Run the following commands in your terminal:

Bash

1. Create a new project directory and navigate into it

```
mkdir jest-testing-lab  
cd jest-testing-lab
```

2. Initialize a new Node.js project (creates package.json)

```
npm init -y
```

3. Install Jest as a development dependency

```
npm install --save-dev jest
```

Configure package.json

Open the generated package.json file in your editor and update the "scripts" section so that npm knows to use Jest for running tests:

JSON

```
{  
  "name": "jest-testing-lab",  
  "version": "1.0.0",  
  "scripts": {  
    "test": "jest"  
  },  
  "devDependencies": {  
    "jest": "^29.0.0"  
  }  
}
```

Level 1: Beginner – Unit Testing Simple Utilities

In this section, you will write pure, deterministic functions and test them using basic Jest matchers.

1. Create the Implementation File

Create a file named `calculator.js`. This file will contain basic arithmetic functions.

JavaScript

```
// calculator.js

function add(a, b) {
  return a + b;
}

function divide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by zero");
  }
  return a / b;
}

module.exports = { add, divide };
```

2. Create the Test File

By convention, Jest looks for files ending in `.test.js`. Create a file named `calculator.test.js`.

JavaScript

```
// calculator.test.js
const { add, divide } = require('./calculator');

describe('Calculator Operations', () => {

  // Test case 1: Addition
  test('adds 2 + 3 to equal 5', () => {
    expect(add(2, 3)).toBe(5);
  });

  // Test case 2: Division
  test('divides 6 by 2 to equal 3', () => {
    expect(divide(6, 2)).toBe(3);
  });

  // Test case 3: Exception Handling
```

```
test('throws an error when dividing by zero', () => {  
  expect(() => divide(10, 0)).toThrow("Cannot divide by zero");  
});
```

3. Execution

Run the test suite using the terminal:

Bash

```
npm test
```

🟡 Level 2: Intermediate – Testing Asynchronous Code & Hooks

Real-world code isn't always synchronous. In this section, we will simulate fetching user data from a database/API using Promises and handle setup/teardown mechanics.

1. Create the Implementation File

Create a file named `userService.js`.

JavaScript

```
// userService.js

// Simulated in-memory database
let users = [];

function clearDatabase() {
  users = [];
}

function fetchUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const user = users.find(u => u.id === id);
      if (user) {
        resolve(user);
      } else {
        reject(new Error("User not found"));
      }
    }, 100); // Simulates network latency
  });
}

function saveUser(user) {
  users.push(user);
}

module.exports = { fetchUser, saveUser, clearDatabase };
```

2. Create the Test File

Create a file named `userService.test.js`. Here, we use `beforeEach` to reset our mock environment state before each individual test runs.

JavaScript

```
// userService.test.js
const { fetchUser, saveUser, clearDatabase } = require('./userService');

describe('User Service (Async Operations)', () => {

  // Lifecycle hook: runs before every single test in this block
  beforeEach(() => {
    clearDatabase();
  });

  test('should resolve with user data when user exists', async () => {
    // Arrange
    const mockUser = { id: 1, name: 'Alice' };
    saveUser(mockUser);

    // Act
    const user = await fetchUser(1);

    // Assert
    expect(user).toEqual(mockUser); // toEqual checks for deep object equality
  });

  test('should reject with an error when user does not exist', async () => {
    // Asserting an asynchronous rejection
    await expect(fetchUser(99)).rejects.toThrow("User not found");
  });
});
```

3. Execution

Run your tests again. Jest will find both test suites automatically.

```
Bash
npm test
```

Summary Checklist for Submission

When evaluating your code or preparing for your lab check-off, verify that your directory output looks clean and all tests pass smoothly:

Target Component	Expected Behavior	Matchers Used
Level 1	Mathematical computations & expected validation edge-cases	toBe(), toThrow()
Level 2	DB Simulation/Asynchronous promises cleanly resolved or rejected	async/await, toEqual()

Make sure you write clean code, handle your edge cases, and ensure your tests pass before pushing your code to repository!